ozon tech
образование

Инфраструктура как код (IaC) под управлением ИИ



Курс по агентской разработке, ИТМО, 2026

ozon tech
образование

О себе

Игорь Поляков

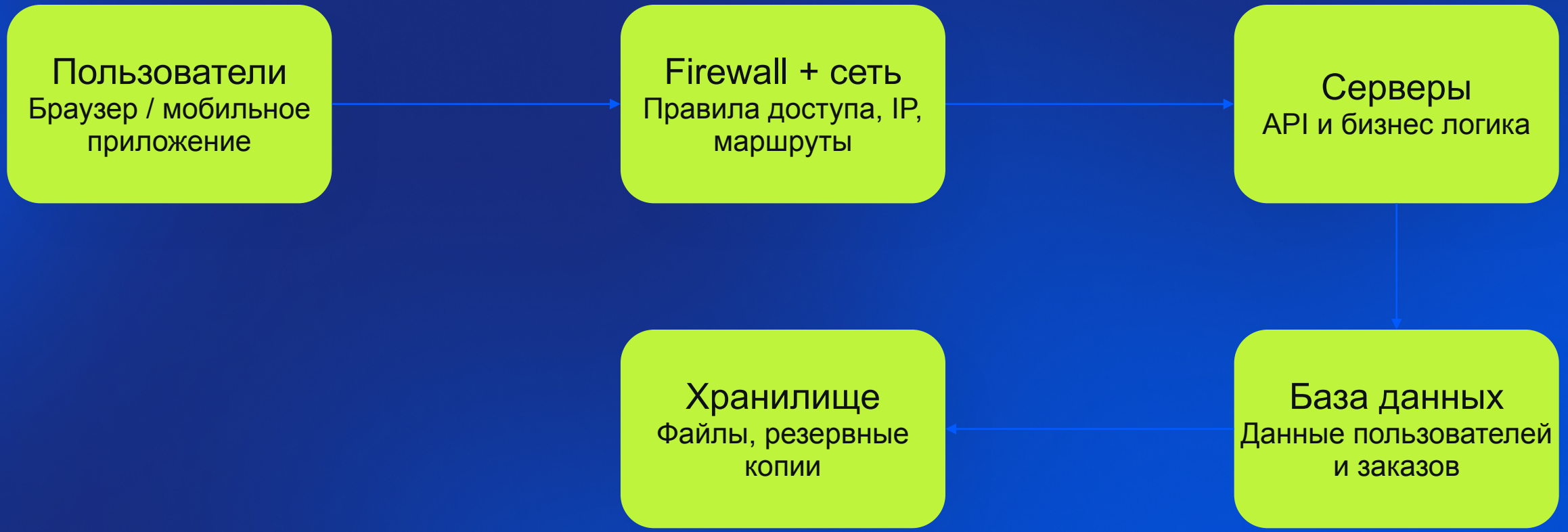
Старший разработчик группы Инфраструктуры данных

Работаю над платформенным векторным поиском



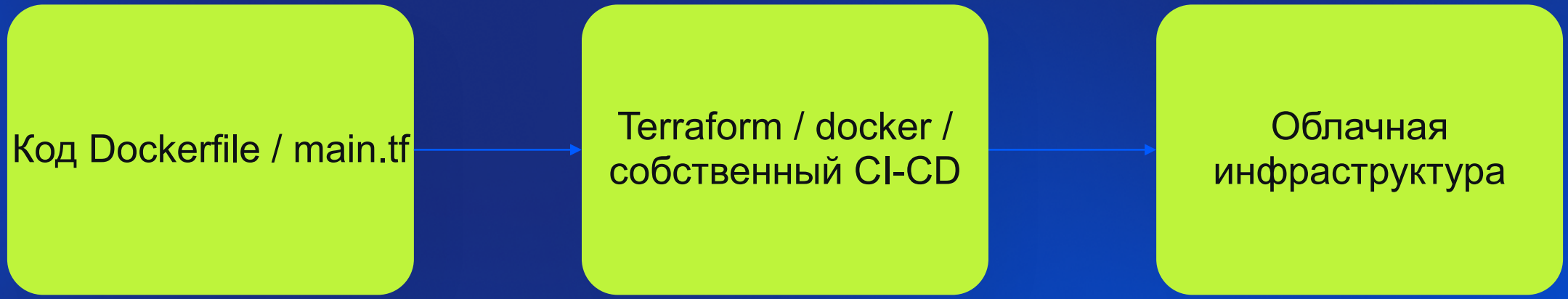
Что такое инфраструктура?

Пример инфраструктуры интернет-сервиса



Что такое Infrastructure as Code?

IaC - это управление инфраструктурой через код, а не через клики в UI.



Идемпотентность
Один и тот же код всегда дает один и тот же результат

Версионность
Код хранится в git, как и приложение

Воспроизводимость
Можно развернуть инфраструктуру с нуля одной командой

Как мы дошли до ИИ: эволюция управления инфраструктурой

2000-е

Ручное управление

SSH, bash-скрипты

2010-е

Инструменты автоматизации

Ansible, Puppet

2014+

Декларативный подход

Terraform, Pulumi

2020+

Модульность и Best Practices

Terraform Modules, Terragrunt

2023+

ИИ Ассистенты

Github Copilot, Container Diet, AI-агенты

Каждый этап решал проблему предыдущего. ИИ решает проблему сложности и времени написания качественного кода.

Я пишу IaC, но хочу писать его быстрее И безопаснее



AI-Ops: Искусственный интеллект в жизненном цикле инфраструктуры

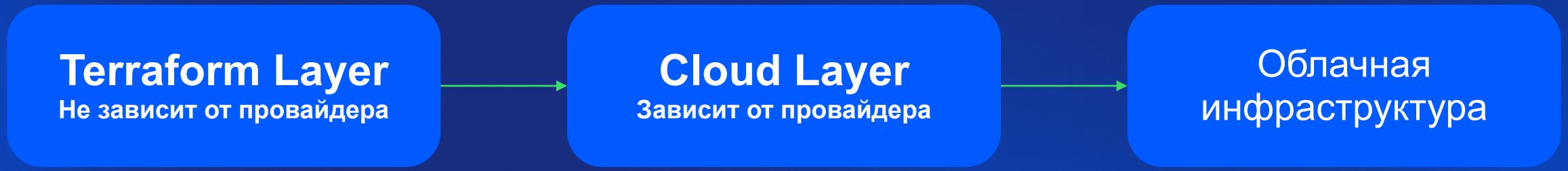
Генерация	Анализ	Оптимизация
Пишет код по описанию	Объясняет terraform plan	Находит дрейф и предлагает фиксы
Предлагает архитектуру	Находит уязвимости	Оптимизирует стоимость
Рефакторит существующий код	Проверяет best practices	Улучшает производительность

Что такое terraform — пример из жизни

Реальный вызов:
У нас есть инфраструктура в Yandex Cloud. Мы решили мигрировать часть сервисов в VK Cloud.

Какой правильный подход?
Как может помочь ИИ?

Terraform — универсальный интерфейс ко всем



- каждый облачный провайдер имеет свой API. Terraform - универсальный интерфейс ко всем.
- ИИ: «Создай мне VPC с подсетью и инстанс с 8 ГБ RAM» -> готовый .tf файл
- ИИ: terraform plan -> человеческий язык «Агент сообщает: Дмитрий пытается увеличить диски на прод-среде, это увеличит бюджет на $50 в месяц. Одобрить?»

Terraform: из чего состоит код

```
provider "yandex" {
  zone = var.zone
}

variable "zone" {
  default = "ru-central1-a"
}

resource "yandex_compute_instance" "web" {
  name = "web-1"
  resources {
    cores  = 2
    memory = 8   # ГБ
  }
  # boot_disk, network_interface …
}

output "id" {
  value = yandex_compute_instance.web.id
}
```

provider
плагин к API облака: Yandex Cloud, VK Cloud, AWS…

resource
объект инфраструктуры: ВМ, сеть, бакет, БД

variable
параметры: зона, размер, окружение

output
что вернуть после apply: ID, IP, URL

Как работает Terraform: init → plan → apply

terraform init
скачивает провайдеры и модули, подключает backend для state

→

terraform plan
сравнивает код с state и реальностью, показывает diff

→

terraform apply
применяет diff через API облака — после подтверждения

terraform destroy
удаляет всё, что описано в коде. В проде — только осознанно

plan ничего не меняет
поэтому его безопасно отдавать ИИ на анализ

Как читать terraform plan

```
Terraform will perform the following actions:

  # yandex_compute_instance.web will be updated in-place
  ~ resource "yandex_compute_instance" "web" {
      ~ resources {
          ~ memory = 8 -> 16
        }
    }

  # yandex_vpc_subnet.old will be destroyed
  - resource "yandex_vpc_subnet" "old" { … }

Plan: 0 to add, 1 to change, 1 to destroy.
```

+ создать

~ изменить на месте

− удалить

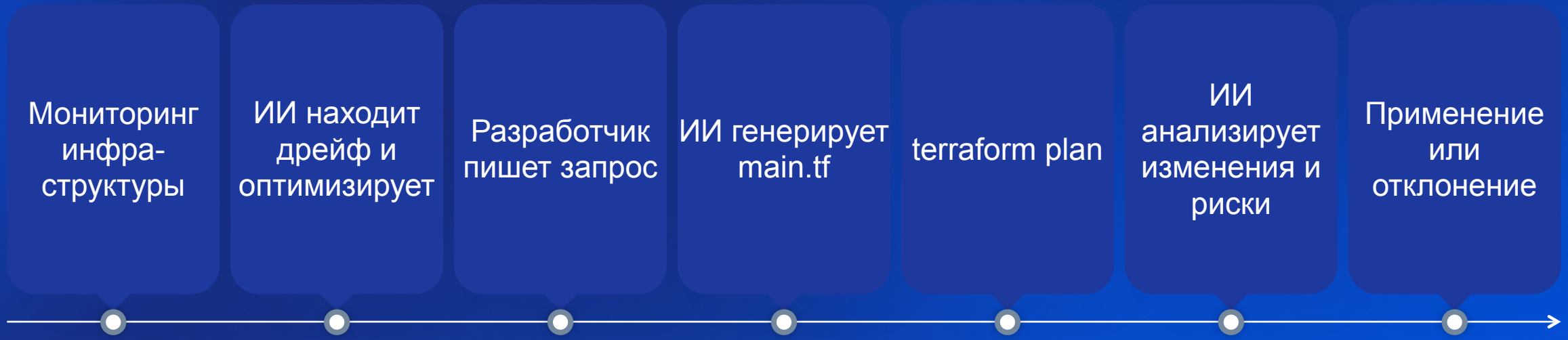
−/+ пересоздать

Самое опасное — «−» и «−/+»: потеря данных и простой. ИИ-ревью подсвечивает их первыми

State: как Terraform помнит инфраструктуру

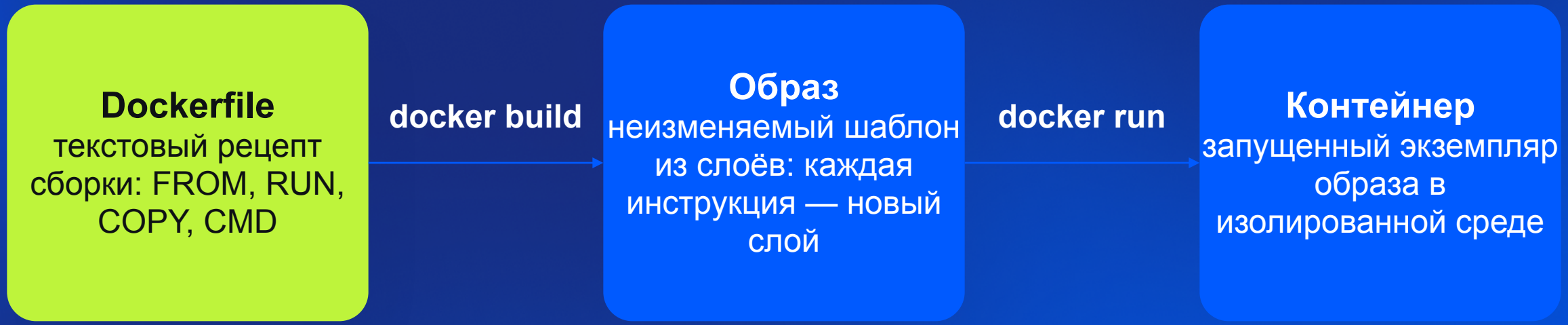


Terraform flow с использованием ИИ агентов



Docker и Dockerfile

Docker упаковывает приложение со всеми зависимостями в образ и запускает его в изолированном контейнере — одинаково на ноутбуке, в CI и в проде



Dockerfile — это тоже IaC

окружение описано кодом: его хранят в git, ревьюят в MR и генерируют с помощью ИИ

01  ИИ в жизненном цикле IaC: генерация, анализ, оптимизация

Генерация кода: наивный промпт

Промпт
«Создай Dockerfile для FastAPI-приложения на Python»

Что сгенерирует ИИ по умолчанию
FROM python:3.9 — тяжёлый образ, 900+ Мб
все зависимости, включая dev
запуск от root
медленная сборка

Генерация кода: хороший промпт

Промпт
«Создай production-grade многоэтапный Dockerfile для FastAPI-приложения на Python с ML-зависимостями»

Что добавит ИИ
многостадийная сборка
предзагрузка ML-моделей на этапе сборки
python:3.11-slim вместо полной версии
non-root пользователь
очистка кеша apt и pip
healthcheck для Kubernetes

Линтер vs AI-ассистент

Традиционный линтинг
синтаксис
простая семантика

AI-ассистент
понимает намерения и объясняет последствия
учитывает контекст изменений
даёт интерактивные рекомендации

❌ Вердикт: ошибка
Настоящий план содержит 3 значительных проблемы в безопасности:
- ACL S3 бакета изменён с приватного на публичный…

Оптимизация структуры модулей: от монолита к композиции

Монолит (до рефакторинга)

```
project/
└── main.tf          # 500+ строк всего подряд
```

terraform plan выполняется минутами

одна ошибка ломает всю инфраструктуру (blast radius)

несколько команд правят один файл — конфликты

Модульная архитектура (после)

```
project/
├── modules/
│   ├── networking/          # VPC, подсети, шлюзы
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   ├── compute/             # EC2/VM инстансы
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   └── database/            # RDS, параметры
│       ├── main.tf
│       ├── variables.tf
│       └── outputs.tf
└── environments/
    ├── dev/
    │   └── main.tf          # вызывает модули с dev-параметрами
    ├── stage/
    │   └── main.tf
    └── prod/
        └── main.tf
```

Автоматическая проверка секретов в коде

Однажды закоммиченный секрет остаётся в истории Git навсегда — даже если код удалён. Поэтому нужна автоматическая проверка.



AI-сканеры секретов: возможности и инструменты

Контекстная проверка
отличает секреты в коде, в комментариях и в тестах

Автофикс
предлагает исправление и может его применить

Интеграция в CI/CD
блокирует коммиты с реальными секретами

Checkov
сканирует IaC (Terraform, K8s) на секреты

GitHub Secret Scanning
на 75% меньше ложных срабатываний

И много других…

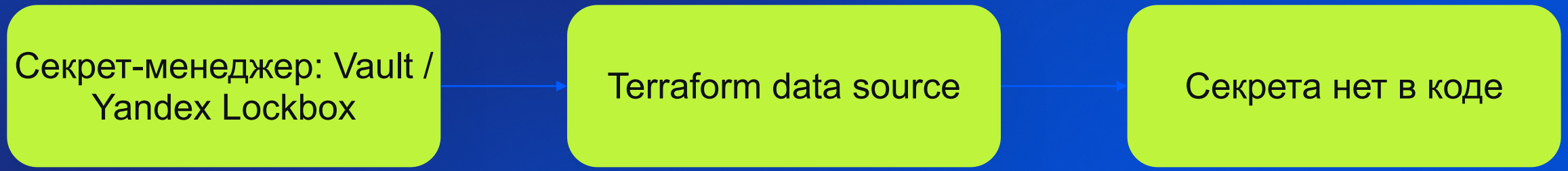
Секреты и чувствительные данные в IaC

Где хранить секреты и как передавать их в Terraform

❌ **Плохо**



✅ **Хорошо**



Секреты и ИИ-агент: главное правило

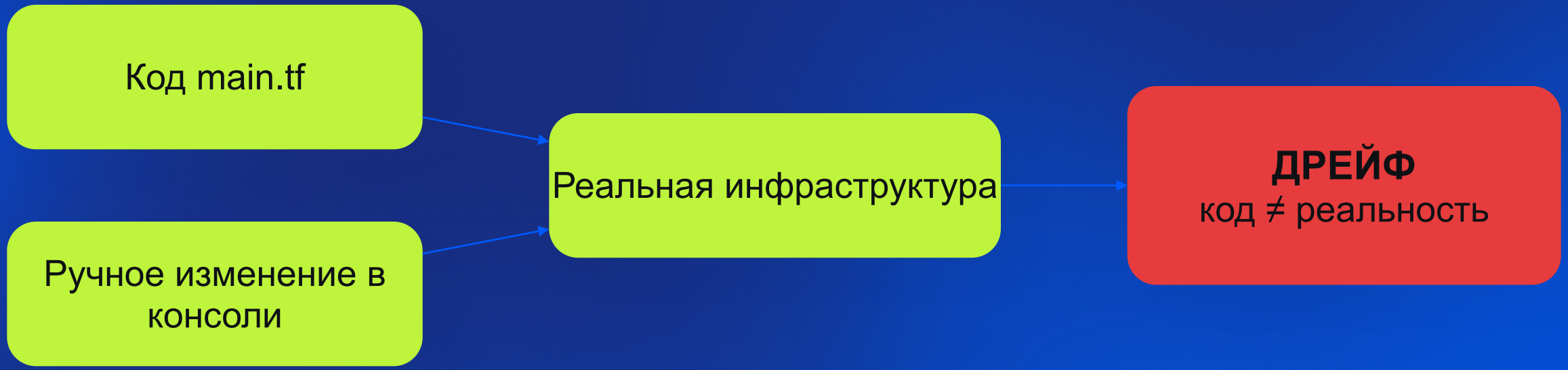
ИИ-агент пишет код интеграции с секрет-менеджером, но сам секрет не видит

Нельзя
хранить секреты в git — даже в private-репозитории, даже в tfvars

Где хранить
HashiCorp Vault, Yandex Lockbox, AWS Secrets Manager, SOPS

Дрейф: код ≠ реальность

Дрейф — «невидимый риск»: код говорит одно, реальность — другое



⚠ **Следующий terraform apply может откатить ручной security-фикс**

кто-то закрыл дыру вручную, а Terraform откроет её обратно

Как ИИ-агент разбирает дрейф



Пример ответа агента на terraform plan -refresh-only

«Порт 22 был открыт для 0.0.0.0/0 вчера в 03:14. Вероятно, это ручной хотфикс во время инцидента»

Стоимость: реагировать или предсказывать

Традиционный подход
счёт вырос — разбираемся через 2–3 недели

AI-driven (Shift Left)
рост бюджета виден до apply: ИИ находит причину и предлагает фикс

Infracost
встраивается в терминал, IDE, ИИ-агента (Claude Code и др.) и CI/CD

Стоимость: как это выглядит в PR



02 Инструменты: ИИ-агенты для IaC

opencode

Model-agnostic агент с открытым исходным кодом, работает в консоли

Что умеет

читает go.mod и структуру проекта
генерирует Dockerfile и .dockerignore
собирает образ и запускает контейнер
анализирует логи

Модели

75+ моделей через Models.dev
DeepSeek V4 Pro — доступна в РФ
Qwen / Kimi — можно запускать локально
локальные модели через Ollama или vLLM
GPT-6 Codex, Claude Sonnet 5 / Opus 5

QwenCode



Вызывает Terraform, Helm и shell-скрипты через встроенный инструмент Shell

Безопасность и качество
защита от разрушительных команд (terraform destroy)
sandbox-режим в отдельном Docker-контейнере
+ обходит OpenCode в FEABench-Ark (синтаксис, компилируемость)
− проиграл OpenCode на своей же модели Qwen в бенче STORE

Модели
Qwen3-Coder-Next
Qwen3-Coder-480B
4 провайдера: Qwen (DashScope), OpenAI, Anthropic, Gemini
оптимизирован под Qwen

MCP: одни и те же серверы во всех агентах

OpenCode

```
{
  "mcp": {
    "terraform": {
      "type": "local",
      "command": ["docker", "run", "-i", "--rm", "hashicorp/terraform-mcp-server:
latest"],
      "enabled": true
    },
    "docker": {
      "type": "local",
      "command": ["npx", "-y", "@hypnosis/docker-mcp-server"],
      "enabled": true
    }
  }
}
```

QwenCode

```
{
  "mcpServers": {
    "terraform": {
      "command": "docker",
      "args": ["run", "-i", "--rm", "hashicorp/terraform-mcp-server:latest"]
    },
    "docker": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-docker"]
    }
  }
}
```

Подключаем одни и те же серверы: Docker MCP Server, Terraform MCP

Claude Code / Codex — стандарт де-факто

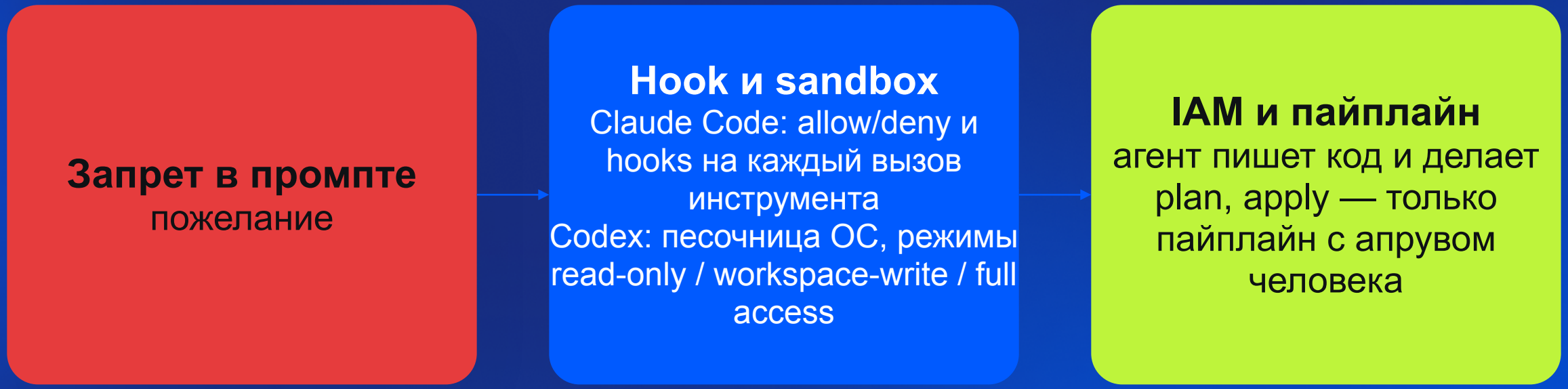
MCP, skills, hooks, субагенты

Инструкции проекта в репозитории: CLAUDE.md / AGENTS.md

Неинтерактивный режим для CI: claude -p, codex exec

Сначала план, потом правки (plan mode)

Гарантии, а не пожелания



Запрет в промпте — это пожелание. Запрет в hook, sandbox или IAM — гарантия.

Container Diet

Узкий open-source инструмент: анализирует Docker-образы и Dockerfile с помощью ИИ

находит раздутые слои

находит проблемы безопасности

подключается к docker-демону

auto-fix

JSON-вывод для CI/CD-пайплайна

работает как MCP-сервер

Модели: 12+ провайдеров ИИ

OpenAI, Anthropic, Ollama (локально), DeepSeek, Groq, OpenRouter…

03  Практика: оптимизация Docker-образов

«Наивный» Dockerfile

```
FROM python:3.11
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "main.py"]
```

Какие здесь есть проблемы?

Проблемы «наивного» Dockerfile

Большой размер
1,2 Гб вместо 200 Мб, избыточные слои

Медленная сборка
любое изменение в коде — переустановка всех зависимостей

Медленный запуск
модели скачиваются при каждом запуске

Низкая безопасность
root-пользователь и уязвимости в зависимостях

В масштабах Ozon
десятки минут простоя при каждом деплое, гигабайты трафика на скачивание образов и риск утечек через уязвимости

Многостадийная сборка: схема



Многостадийная сборка: Dockerfile

```
# Stage 1: Build
FROM golang:1.21-alpine AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -ldflags="-s -w" -o /out/service ./cmd/server

# Stage 2: Runtime
FROM scratch
COPY --from=builder /out/service /service
CMD ["/service"]
```

Многостадийная сборка: эффект

~300 Мб

без multi-stage: компилятор Go, SDK, исходники, бинарник

в 20+ раз меньше

~15 Мб

с multi-stage: только бинарник

Пример промпта

«У меня Go-микросервис с gRPC. Сделай multi-stage Dockerfile с минимальным runtime-образом. Используй scratch, статическую линковку и оптимизацию размера бинарника»

Какой базовый образ выбрать

Образ	Когда использовать	Размер
scratch	Go, Rust - статические бинарники	~0 Мб
distroless	Go, Java - нужны CA-сертификаты	~2-20 Мб
alpine	Python, Node.js - нужен shell	~5 Мб
slim	Python, Node.js - нужны системные библиотеки	~50-150 Мб

Почему для Go часто используют scratch, а для Python — slim?

Минимизация слоев

Каждая инструкция RUN, COPY, ADD — новый слой. Удаление в следующем слое место не освобождает

```
FROM python:3.11-slim
WORKDIR /app

RUN apt-get update
RUN apt-get install -y gcc g++ make
RUN pip install --no-cache-dir -r requirements.txt
RUN apt-get remove -y gcc g++ make
RUN apt-get clean
RUN rm -rf /var/lib/apt/lists/*

COPY . .
CMD ["python", "main.py"]
```

RUN apt-get update

RUN apt-get install -y gcc g++ make

RUN pip install …requirements.txt

RUN apt-get remove -y gcc g++ make

RUN apt-get clean

RUN rm -rf /var/lib/apt/lists/*

Удаляй то, что скачал, в той же команде RUN, где скачал

Минимизация слоев

```
RUN apt-get update && apt-get install -y --no-install-recommends \
        gcc g++ make \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

все apt-операции в одном RUN через &&

очистка кеша apt в той же команде

--no-install-recommends: без лишних пакетов

--no-cache-dir для pip: без кеша

Промпт для ИИ
«Оптимизируй этот Dockerfile: объедини RUN-команды, добавь очистку кеша в те же слои, используй --no-install-recommends и --no-cache-dir»

+ комбинируй с multi-stage

Управление модельными артефактами

Веса и токенизаторы — самая «тяжёлая» часть AI-сервиса: от места их загрузки зависят холодный старт и стабильность

❌ Наивный подход

Контейнер стартует → Скачивает модель из HuggingFace → ⏱️ Холодный старт: 30 с – 10 мин → ⚠️ Риски: таймауты, недоступность сети

✅ Правильный подход

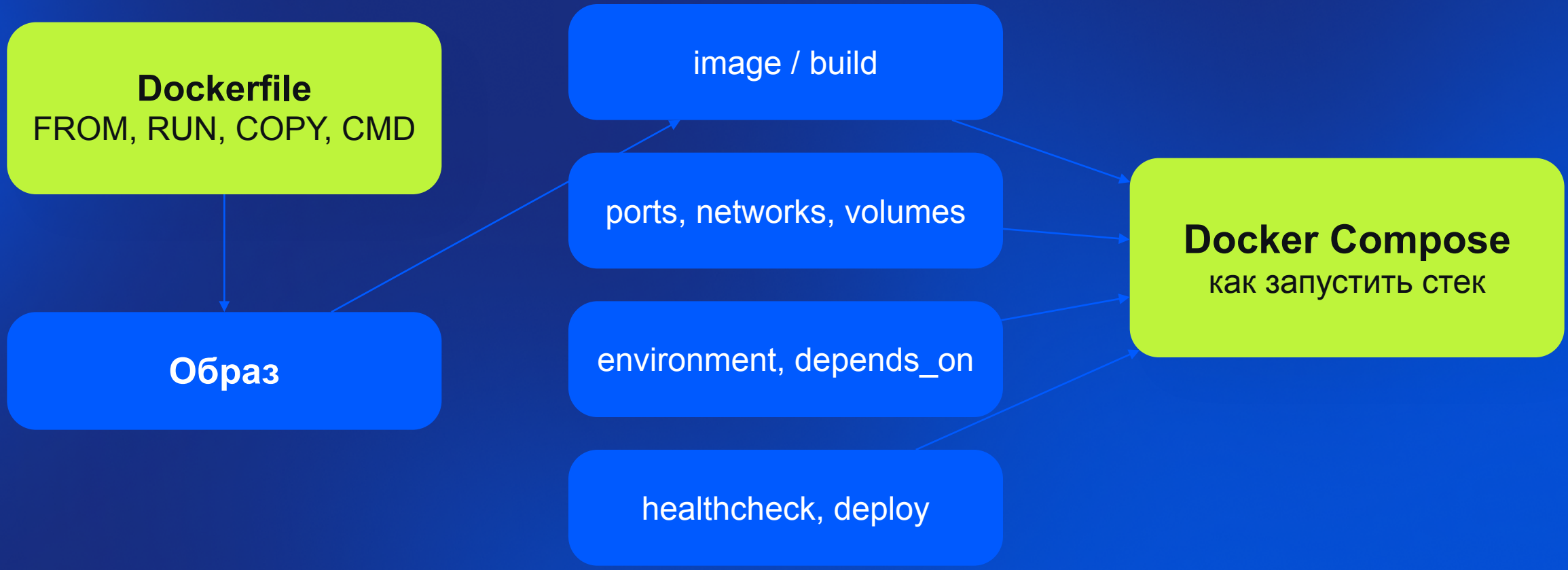
Сборка образа (docker build) → RUN python -c: скачать модель в кеш → 🚀 Холодный старт: 2–5 с → ✅ Работает офлайн и предсказуемо

Как ещё уменьшить модельные артефакты

Образ	Когда использовать	Размер
Предзагрузка модели	Холодный старт с минут до секунд	RUN python -c «AutoModel.from_pretrained(…)»
Выбор формата	Экономия 200-300 Мб	ONNX Runtime вместо PyTorch
CPU-only PyTorch для linux	Экономия 1.5 Гб	pip install torch —index-url ../cpu

Docker Compose: от образа к стеку

Dockerfile собирает образ, Docker Compose — стек из нескольких образов на одной машине (в режиме swarm — на кластере)



Роли ИИ в Docker Compose

Генерация по описанию
полный YAML с healthcheck и сетевыми настройками

Анализ существующего Compose
проблемы с безопасностью, производительностью и зависимостями

Оптимизация и рефакторинг
YAML anchors, быстрые healthcheck, логирование с ротацией

Инструменты для Docker Compose

Инструмент	Роль	Особенность	Инструмент	Роль	Особенность
RDT (Rambo Docker Toolkit)	CLI-визард для генерации Compose	rdt add postgres —yes добавляет сервис с дефектными настройками	Docker Manager Plugin	Субаренды для Claude Code	Compose-generator, resource-optimizer
NeptureAI	Генерация Compose по естественному языку	Fine-tuned модель на RWKV для Dockerfile, compose, Github Workflows	Docker MCP Server	MCP-сервер для агентов	Управление Compose-проектами, health-диагностика, логи

Стоимость владения AI-агентами в IaC

Модель	Стоимость за 1M токенов	Когда использовать
Локальная (Ollama)	0 (только железо)	Изоляция данных, простые задачи
DeepSeek	~$0.14 / $0.28	Массовая генерация, анализ
Claude Sonnet	~$3 / $15	Сложные архитектурные задачи
GPT-6	~$10 / $30	Критичные задачи

Для IaC не нужна самая дорогая модель

задачи структурные, не творческие: агент может сам выбирать модель под задачу

Ограничения ИИ

Что ИИ не умеет, что делает плохо и на что обратить внимание

Забывает контекст
детали теряются по мере роста контекста

Может ошибаться даже в простом, например - в арифметике
особенно во вспомогательных вычислениях

Делает только костяк
доводить до ума и полировать — уже ваша задача

Чек-лист: где ИИ помогает, а где — нет

❌ ИИ не заменит человека

- Решение о том, ЧТО разворачивать
- Аудит безопасности критичных систем
- Ответственность за прод-инциденты
- Понимание бизнес-контекста

✅ ИИ отлично справляется

- Генерация Dockerfile / Terraform
- Объяснение terraform plan на человеческом языке
- Поиск синтаксических и семантических ошибок
- Рефакторинг

Итог

ИИ — это усилитель, а не замена
он убирает рутину, но не принимает решений

В Ozon Tech мы используем ИИ для ускорения, но approval остаётся за инженерами

ozon tech
образование

Спасибо за внимание. Вопросы?



Если остались вопросы, пиши на @igor_polyakov